



计算机程序设计L习题课

时间:

2025-12-14

by: 杜晓阳 宋雨声 喻云山



实验内容回顾：

实验3：P36 2数据类型的大小

```
zhangsan() { //抄写书上代码，完成填空，得到与输入输出样例相符的结果
    char c = 'a';
    int i = 6;
    float f = 1.23456789; //由于有效位数限制，实际存储的值并不是1.23456789
    double d = 1.23456789; //double类型的有效位数足够存储1.23456789
    printf("char常量/变量/类型占用字节数:%d,%d,%d\n", sizeof('a'), sizeof(c), sizeof(char));
    printf("int常量/变量/类型占用字节数:%d,%d,%d\n", sizeof(123), sizeof(i), sizeof(int));
    printf("float常量/变量/类型占用字节数:%d,%d,%d\n", sizeof(1.2345f), sizeof(f), sizeof(float)); /*浮点型常量默认是double类型,需要加上f后缀才能视为float类型*/
    printf("double常量/变量/类型占用字节数:%d,%d,%d\n", sizeof(1.2345), sizeof(d), sizeof(double));

    printf("表达式++i占用存储空间字节数:%d\n", sizeof(++i));
    printf("表达式c=++i占用存储空间字节数:%d\n", sizeof(c=++i)); //超出有效位的数无效
    printf("i的值为%d\n", i);
    printf("f的值为%.10f\n", f);
    printf("d的值为%.10f\n", d); //不足位数用0补齐

    printf("表达式d+c+i占用存储空间字节数:%d\n", sizeof(d+c+i));
    printf("自指定表达式占用的字节数:%d\n", sizeof(i+f+d)); /*自行使用前面定义的至少3个变量组成一个表达式*/

    return 0;
}
```



实验内容回顾：

实验3：P36 2数据类型的大小

sizeof运算符的作用

sizeof是C语言的一个编译时运算符（不是函数），用于计算数据类型或表达式结果类型所占的字节数。

sizeof('a'): 字符常量'a'在C中是int类型，所以输出4字节（32位系统）

sizeof(c): char变量c，输出1字节

sizeof(char): char类型，输出1字节

sizeof(123): 整型常量，通常为4字节

sizeof(i): int变量i，4字节

sizeof(int): int类型，4字节

第2题

char常量/变量/类型占用字节数:4, 1, 1

int常量/变量/类型占用字节数:4, 4, 4

float常量/变量/类型占用字节数:4, 4, 4

double常量/变量/类型占用字节数:8, 8, 8

表达式++i占用存储空间字节数:4

表达式c=++i占用存储空间字节数:1

i的值为6

f的值为1.2345678806

d的值为1.2345678900

表达式d+c+i占用存储空间字节数:8

自指定表达式占用的字节数:8

sizeof(1.2345f): float类型常量（有f后缀），4字节

sizeof(f): float变量f，4字节

sizeof(float): float类型，4字节

实验内容回顾：

实验3：P36 2数据类型的大小

sizeof运算符的作用

sizeof是C语言的一个编译时运算符（不是函数），用于计算数据类型或表达式结果类型所占的字节数。

sizeof(1.2345): 浮点常量默认是double类型，8字节

sizeof(d): double变量d，8字节

sizeof(double): double类型，8字节

sizeof(++i): 表达式++i的结果类型是int，所以是4字节

重要：sizeof不会计算表达式，只检查类型，所以i的值不会增加

`printf("表达式d+c+i占用存储空间字节数:%d\n", sizeof(d+c+i));`表达式d+c+i的类型转换：char→int→double，最终为double类型，8字节

`printf("自指定表达式占用的字节数:%d\n", sizeof(i+f+d));`表达式i+f+d的类型：int+float→float，然后float+double→double，最终为double类型，8字节

第2题

```
char常量/变量/类型占用字节数:4, 1, 1
int常量/变量/类型占用字节数:4, 4, 4
float常量/变量/类型占用字节数:4, 4, 4
double常量/变量/类型占用字节数:8, 8, 8
表达式++i占用存储空间字节数:4
表达式c=++i占用存储空间字节数:1
i的值为6
f的值为1.2345678806
d的值为1.2345678900
表达式d+c+i占用存储空间字节数:8
自指定表达式占用的字节数:8
```

sizeof(c=++i): 表达式c=++i的结果类型是char（赋值表达式类型是左操作数类型），所以是1字节

同样，表达式不会实际执行，c和i的值不变

注释"超出有效位的数无效"应指字符变量只能存储有限范围的值

实验内容回顾：

实验3：P36 2数据类型的大小

关键知识点总结：

1. sizeof是运算符，不是函数：在编译时确定大小，不计算表达式

2. 类型提升规则：

- 字符常量'a'是int类型
- 浮点常量默认是double类型
- 表达式中，char、short会自动提升为int
- 表达式中，float会自动提升为double

3. 精度问题：

- float：约6-7位有效数字
- double：约15-16位有效数字

4. 格式说明符：

- 使用%.10f控制浮点数输出精度

```
1 #include <stdio.h>
2
3 int main() {
4     int i = 10;
5
6     printf("最简单的验证: \n");
7     printf("sizeof(++i) = %zu\n", sizeof(++i));
8     printf("但i的值仍然是: %d\n", i);
9     printf("结论: sizeof不计算表达式! \n");
10
11     return 0;
12 }
```

C:\Users\dxydu\Desktop\计算机程序设计\实验\实验3\第3次实验参考代码\第3次实验参考代码\3333.exe

最简单的验证:
sizeof(++i) = 4
但i的值仍然是: 10
结论: sizeof不计算表达式!

Process exited after 0.4339 seconds with return value 0
请按任意键继续. . .



实验内容回顾：

```
#include <stdio.h>
#include <stdlib.h>
```

No7() // 第7题, P42 6逻辑表达式, 提示: 这个题目采用 关系运算符 和 逻辑运算符 以及 自增自减运算符 来完成

```
int i, res;
printf("请输入一个整数: ");
scanf("%d", &i); // 输入一个整数到i

// 第一个表达式: 如果i为0, 则i自增1; 否则i不变。
// 当i为0时, !i为真(1), 继续计算&&右边, ++i使i变为1。res结果为 1 && 1 -> 1。
// 当i不为0时, !i为假(0), &&发生短路, 右边不计算, i不变。res结果为 0。
res = !i && ++i;
printf("&&=%d,i=%d\n", res, i);

// 第二个表达式: 如果i不为1, 则i自减1; 否则i不变。
// 当i不为1时, (i!=1)为真(1), 继续计算&&右边, --i使i减1。res结果为 1 && (非零) -> 1。
// 当i为1时, (i!=1)为假(0), &&发生短路, 右边不计算, i不变。res结果为 0。
res = (i != 1) && --i;
printf("||=%d,i=%d\n", res, i);

// 第三个表达式: i为1或2, 或res为0时, 结果为1。
// 这是一个纯逻辑判断, 将条件直接用逻辑或||连接起来即可。
res = (i == 1) || (i == 2) || (res == 0);
printf("res=%d\n", res);

return 0;
```

i++ // 后置自增: 先使用i的值, 再自增
++i // 前置自增: 先自增, 再使用i的值
i-- // 后置自减: 先使用i的值, 再自减
--i // 前置自减: 先自减, 再使用i的值

if (x = 5) // 错误! 这是赋值, 永远为真
if (x == 5) // 正确! 这是比较

&& (逻辑与) - 两个条件都为真时返回真
|| (逻辑或) - 至少一个条件为真时返回真
! (逻辑非) - 条件为假时返回真, 条件为真时返回假

短路求值特性:

- &&: 第一个为假, 不再计算第二个
- ||: 第一个为真, 不再计算第二个



3.2.4 表达式的运算次序

包含两个以上运算符的表达式需要通过优先级与结合性（也称结合律、结合方向）规则确定运算符的运算次序。表 3-2 给出了除位运算符以外所有运算符的优先级与结合性，同一行中的运算符具有相同的优先级，各行间从上往下优先级逐行降低。完整的运算符优先级与结合性列表见附录 C。

表 3-2 运算符的优先级与结合性

优先级	运算符	结合性
1	() [] -> . 后缀++ 后缀--	从左向右
2	! ~ ++ -- + - * & (type) sizeof 前缀++ 前缀--	从右向左
3	* / %	从左向右
4	+ -	
6	< <= > >=	
7	== !=	
11	&&	
12		从右向左
13	?:	
14	= += -= *= /= %=	从左向右
15	,	

在大的类别上，一元运算符优先级 > 二元运算符；算术运算符优先级 > 关系运算符 > 逻辑运算符 > 赋值运算符。同类运算符中也有优先级的差异，如乘、除高于加、减，逻辑与高于逻辑或。

表达式中若有多个相同优先级的运算符，则按结合性确定计算次序。结合性分为从右向左和从左向右两种结合方向。除了后缀自增自减，一元运算符都只能从右向左结合；除了赋值运算符，二元运算符都是从左向右结合，即优先级相同时，按从左向右的顺序计算。这与数学中的规定类似。唯一的三元运算符则从右向左结合。

```
2 #include <math.h> // 需要包含这个头文件来使用fabs函数
3
4 int main() {
5     // 定义浮点数a
6     float a = 0.1 + 0.2;
7
8     printf("浮点数比较演示\n");
9     printf("=====\n\n");
10
11     // 显示a的实际值
12     printf("a = 0.1 + 0.2\n");
13     printf("a的实际值: %.20f\n", a);
14
15     // 错误方法: 直接使用==比较
16     printf("错误方法: if (a == 0.3)\n");
17     if (a == 0.3) {
18         printf(" 结果: 相等\n");
19     } else {
20         printf(" 结果: 不相等\n");
21     }
22     printf(" (浮点数有精度问题, 直接比较不可靠) \n\n");
23
24     // 正确方法: 使用fabs和容差比较
25     printf("正确方法: if (fabs(a - 0.3) < 1e-6)\n");
26     printf(" fabs(a - 0.3) = %.20f\n", fabs(a - 0.3));
27     printf(" 1e-6 = 0.000001\n");
28
29     if (fabs(a - 0.3) < 1e-6) {
30         printf(" 结果: 在误差范围内相等 ?\n");
31     } else {
32         printf(" 结果: 不在误差范围内 ?\n");
33     }
34
35     printf("\n程序说明: \n");
36     printf("1. 浮点数在计算机中以二进制存储, 有精度限制\n");
37     printf("2. 0.1和0.2无法用二进制精确表示\n");
38     printf("3. 因此0.1+0.2的结果不是精确的0.3\n");
39     printf("4. 正确做法是比较差值是否小于一个很小的数\n");
40
41     return 0;
42 }
```

C:\Users\dxydu\Desktop\333.exe

浮点数比较演示

=====

a = 0.1 + 0.2

a的实际值: 0.30000001192092896000

错误方法: if (a == 0.3)

结果: 不相等

(浮点数有精度问题, 直接比较不可靠)

正确方法: if (fabs(a - 0.3) < 1e-6)

fabs(a - 0.3) = 0.0000001192092896618

1e-6 = 0.000001

结果: 在误差范围内相等 ?

程序说明:

1. 浮点数在计算机中以二进制存储, 有精度限制
2. 0.1和0.2无法用二进制精确表示
3. 因此0.1+0.2的结果不是精确的0.3
4. 正确做法是比较差值是否小于一个很小的数

Process exited after 0.4673 seconds with return value 0

请按任意键继续. . .


```
1  #include <stdio.h>
2
3  int main() {
4      char ch;
5      while (1) { // 无限循环, 直到输入'0'退出
6          ch=getchar();
7          if (ch == '0') { // 输入'0'时退出循环
8              break;
9          }
10         if (ch == '\n') { // 跳过换行符 (避免将回车误判为输入)
11             continue;
12         }
13
14         // 判断是否为小写字母
15         if (ch >= 'a' && ch <= 'z') {
16             printf("%c\n", ch - 'a' + 'A'); // 小写转大写并换行输出
17         }
18         // 判断是否为大写字母
19         else if (ch >= 'A' && ch <= 'Z') {
20             printf("yes\n"); // 输出yes并换行
21         }
22         // 非字母情况
23         else {
24             printf("no\n"); // 输出no并换行
25         }
26     }
27     return 0;
28 }
```



使用C标准库函数 (来自ctype.h) :

- islower(ch) // 判断是否小写字母
- isupper(ch) // 判断是否大写字母
- isdigit(ch) // 判断是否数字
- isspace(ch) // 判断是否空白字符
- isalpha(ch) // 判断是否字母
- isalnum(ch) // 判断是否字母或数字
- ch - 'a' + 'A' // 小写转大写
- ch - 'A' + 'a' // 大写转小写
- ch - '0' // 字符数字转整数数字

toupper(ch) // 小写转大写
tolower(ch) // 大写转小写



```
1  #include <stdio.h>
2
3  int main() {
4      char ch;
5      while (1) { // 无限循环, 直到输入'0'退出
6          ch=getchar();
7          if (ch == '0') { // 输入'0'时退出循环
8              break;
9          }
10         if (ch == '\n') { // 跳过换行符 (避免将回车误判为输入)
11             continue;
12         }
13
14         // 判断是否为小写字母
15         if (ch >= 'a' && ch <= 'z') {
16             printf("%c\n", ch - 'a' + 'A'); // 小写转大写并换行输出
17         }
18         // 判断是否为大写字母
19         else if (ch >= 'A' && ch <= 'Z') {
20             printf("yes\n"); // 输出yes并换行
21         }
22         // 非字母情况
23         else {
24             printf("no\n"); // 输出no并换行
25         }
26     }
27     return 0;
28 }
```

break语句

用途：立即终止当前循环或switch语句

执行后：程序跳出当前循环/switch，继续执行之后的代码

continue语句

用途：跳过当前循环的剩余部分，立即开始下一次循环

执行后：循环中continue之后的代码不再执行，但循环继续



```

1  #include <stdio.h>
2
3  int main() {
4      int digits[] = {5, 6, 7, 8}; // 可用的数字
5      int count = 0; // 计数器, 用于控制每行输出的个数
6
7      // 三层循环分别生成百位、十位、个位数字
8      for (int i = 0; i < 4; i++) {          // 百位数字
9          for (int j = 0; j < 4; j++) {      // 十位数字
10             if (j == i) continue;          // 十位与百位不能相同
11             for (int k = 0; k < 4; k++) {   // 个位数字
12                 if (k == i || k == j) continue; // 个位与百位或十位不能相同
13
14                 // 计算当前三位数的值
15                 int number = digits[i] * 100 + digits[j] * 10 + digits[k];
16                 printf("%d ", number);
17
18                 count++;
19                 // 每输出4个数换行
20                 if (count % 4 == 0) {
21                     printf("\n");
22                 }
23             }
24         }
25     }
26
27     return 0;
28 }

```

```

for (int i = 0; i < 4; i++) {          // 外层循环：百位
    for (int j = 0; j < 4; j++) {      // 中层循环：十位
        for (int k = 0; k < 4; k++) { // 内层循环：个位

```

if (j == i) continue; // 跳过十位=百位的情况
 if (k == i || k == j) continue; // 跳过个位=百位或个位=十位的情况
 用于跳过重复数字的组合

当条件满足时，跳过当前循环的剩余部分，直接开始下一次循环

避免数字重复，确保三位数的各个位数都不相同



```
1 #include <stdio.h>
2
3 int main() {
4     //printf("所有的水仙花数有: \n");
5
6     // 遍历所有三位数
7     for (int num = 100; num <= 999; num++) {
8         int original = num;
9         int sum = 0;
10
11         // 分解数字并计算立方和
12         while (original != 0) {
13             int digit = original % 10;
14             sum += digit * digit * digit;
15             original /= 10;
16         }
17
18         // 判断是否为水仙花数
19         if (sum == num) {
20             printf("%d ", num);
21         }
22     }
23
24     printf("\n");
25     return 0;
26 }
```

```
while (original != 0) {
    int digit = original % 10; // 获取个位数
    sum += digit * digit * digit; // 计算立方和
    original /= 10; // 去掉个位
}
```

工作流程示例 (以153为例) :

循环次数	original	digit (original%10)	计算digit ³	更新sum	original/=10
第1次	153	3	3 ³ =27	0+27=27	15
第2次	15	5	5 ³ =125	27+125=152	1
第3次	1	1	1 ³ =1	152+1=153	0



```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()//找零问题
5  {
6      float cost=0;
7      int changes=0;
8      int cat[]={500,200,100,50,10,5,1};
9      int i=0,cnt=0;
10
11     printf("请输入花费（单位：元）：");
12     scanf("%f",&cost);
13     if(cost < 100 && cost > 0)
14     {
15         changes = (100-cost+0.05)*10;
16         printf("\n找零%.1f元，方案：",changes/10.0);
17         for(i=0;i<7;i++)
18         {
19             cnt = 0;
20             while(changes>=cat[i])
21             {
22                 changes-=cat[i];
23                 cnt++;
24             }
25             if(cnt > 0)
26                 printf("%.1f元%d张 ",cat[i]/10.0,cnt);
27         }
28         printf("\n");
29         return 0;
30     }
```

// 代码方法

$\text{changes1} = (100 - \text{cost} + 0.05) * 10;$

// 标准四舍五入方法

$\text{changes2} = (\text{int})((100 - \text{cost}) * 10 + 0.5);$



```
while(fabs(x1-x2)>=1e-6)
{
    x1=x2;
    fx1= 3*x1*x1*x1-3*x1*x2+x1-6; // f(x)
    fx2=9*x1*x1-6*x1+1;           // f'(x) 导数
    x2=x1-fx1/fx2;                 // 牛顿迭代公式
    cnt1++;
}
```

$x_{n+1} = x_n - f(x_n)/f'(x_n)$
算法步骤：

选择初始近似值 x_0

计算 $f(x_n)$ 和 $f'(x_n)$

更新： $x_{n+1} = x_n - f(x_n)/f'(x_n)$

重复直到满足精度要求

```
while(fabs(x1-x2)>=1e-6)
{
    double m=(x1+x2)/2;
    if((3*m*m*m-3*m*m+m-6)*fx2<0)
        x1=m;
    else
        x2=m;
    cnt2++;
}
```

二分法原理：

- 1.在区间 $[a,b]$ 上，如果 $f(a) \cdot f(b) < 0$ ，则区间内必有根
- 2.取中点 $c = (a+b)/2$
- 3.判断 $f(c)$ 的符号，替换 a 或 b
- 4.重复直到区间足够小



2.1 结构体（Struct）

```
struct student {  
    int gid;      // 编号  
    char name[20]; // 姓名  
    char gender;  // 性别:M或F  
    float score;  // 成绩  
};
```

关键点：

定义结构体类型：struct student是一种新的数据类型

成员变量：不同类型的变量组合在一起

内存布局：结构体成员在内存中连续存储

访问方式：使用.运算符访问成员，如a[i].gid

2.2 文件操作（File Operations）

```
fpi = fopen("stardata.txt", "r"); // 只读模式  
fpo = fopen("stardata.txt", "a"); // 追加模式  
文件模式：
```

"r"：只读，文件必须存在

"w"：只写，创建新文件或覆盖已有

"a"：追加，在文件末尾添加

"r+"：读写，文件必须存在

"w+"：读写，创建新文件

"a+"：读写追加

读取文件内容

```
while(!feof(fpi)) {    // 判断是否到达文件末尾
    putchar(fgetc(fpi)); // 逐个字符读取并输出
}相关函数：
```

feof()：检查文件结束标志

fgetc()：读取一个字符

putchar()：输出一个字符到屏幕

```
写入文件fprintf(fpo, "%d-%02d-%02d,%02d:%02d:%02d\n",
    stm->tm_year + 1900, stm->tm_mon + 1,
stm->tm_mday,
    stm->tm_hour, stm->tm_min, stm-
>tm_sec);
```

[C语言文件指针，如何对文件进行操作，文件指针FILE，指向文件指针-CSDN博客](#)

struct tm结构体的定义

struct tm 结构体的定义如下：

```
struct tm {
    int tm_sec; // 秒，范围从0到59
    int tm_min; // 分，范围从0到59
    int tm_hour; // 小时，范围从0到23
    int tm_mday; // 一个月中的日期，范围从1到31
    int tm_mon; // 月份，范围从0到11（0表示一月）
    int tm_year; // 年份，其值等于实际年份减去1900
    int tm_wday; // 星期，范围从0到6（0表示星期天）
    int tm_yday; // 一年中的第几天，范围从0到365
    int tm_isdst; // 夏令时标识符
};
```

在C/C++中，`struct tm`是一个用于处理日期和时间的结构体。它定义在`time.h`头文件中，包含了年、月、日、小时、分钟、秒等时间元素。`struct tm`结构体通常与`time_t`类型一起使用，`time_t`是一个表示时间的长整型数，用于存储自1970年1月1日以来的秒数。

```
seconds = time(NULL); // 填空位置9：获取当前时间秒数
    stm = localtime(&seconds); // 填空位置10：转换为
本地时间结构体
    fprintf(fpo, "%d-%02d-%02d,%02d:%02d:%02d\n",
        stm->tm_year + 1900, stm->tm_mon + 1, stm-
>tm_mday,
        stm->tm_hour, stm->tm_min, stm->tm_sec); //
填空位置11：写入时间戳
```




```
struct student {  
    int gid;      // 编号  
    char name[20]; // 姓名  
    char gender;  // 性别:M或F  
    float score;  // 成绩  
};
```

struct是关键字，表示定义结构体

student是结构体标签（tag）

大括号内是结构体成员

结构体定义以分号结束

定义结构体实际上是定义了一种新的数据类型，
就像int、float一样。

```
// 方式1：先定义结构体，再声明变量  
struct student stu1; // 声明一个结构体变量
```

```
// 方式2：定义结构体时声明变量  
struct student {  
    int gid;  
    char name[20];  
} stu2, stu3; // 同时声明stu2和stu3
```

```
// 方式3：匿名结构体（不推荐）  
struct {  
    int id;  
    char name[20];  
} stu4; // 只能使用这一次
```

结构体数组

```
struct student a[100]; // 声明包含100个学生结构体的数  
组
```



结构体成员的访问

点运算符 (.)

// 访问单个结构体变量的成员

```
struct student stu;
```

```
stu.gid = 1001;          // 赋值
```

```
strcpy(stu.name, "张三"); // 字符串需要复制
```

```
stu.gender = 'M';
```

```
stu.score = 88.5;
```

// 读取成员值

```
int id = stu.gid;
```

```
float score = stu.score;
```

箭头运算符 (->)

// 通过指针访问结构体成员

```
struct student stu;
```

```
struct student *p = &stu; // 指向结构体的指针
```

```
p->gid = 1002;    // 等价于 (*p).gid
```

```
strcpy(p->name, "李四");
```

```
p->gender = 'F';
```

```
p->score = 92.0;
```



整数部分转换 (除2取余法)

```
while (t_i > 0 && len_i < 128) {  
    b_i[len_i++] = t_i % 2; // 取余数  
    t_i = t_i / 2;         // 除以2  
}
```

小数部分转换 (乘2取整法)

```
while (t_f > 1e-6 && j < 32) {  
    t_f *= 2;             // 乘以2  
    b_f[j] = (int)t_f;     // 取整数部分  
    t_f -= b_f[j];         // 减去整数部分  
    j++;  
}
```

数值分离:

```
t_i = (unsigned long long)x; // 整数部分  
t_f = x - t_i;              // 小数部分
```



美国数学家维纳智力早熟，11岁就上了大学。一次，他参加某个重要会议，他年轻的面孔引入注目，于是有人询问他的年龄。他回答说：“我年龄的立方是一个4位数，我年龄的4次方是一个6位数，这10个数字正好包含了0~9这10个数字，每个数字还都恰好出现1次。”请编程计算他当时的年龄。

维纳年龄需要满足三个条件：

年龄的立方是4位数

年龄的四次方是6位数

立方和四次方的10个数字正好包含0-9，每个数字出现一次

数学推导确定年龄范围

立方是4位数： $1000 \leq x^3 \leq 9999 \Rightarrow 10 \leq x \leq 21$
(因为 $10^3=1000$ ， $21^3=9261$ ， $22^3=10648>9999$)

四次方是6位数： $100000 \leq x^4 \leq 999999 \Rightarrow 18 \leq x \leq 31$
(因为 $18^4=104976$ ， $31^4=923521$)

取交集： $18 \leq x \leq 21$

美国数学家维纳智力早熟，11岁就上了大学。一次，他参加某个重要会议，他年轻的面孔引入注目，于是有人询问他的年龄。他回答说：“我年龄的立方是一个4位数，我年龄的4次方是一个6位数，这10个数字正好包含了0~9这10个数字，每个数字还都恰好出现1次。”请编程计算他当时的年龄。

```
bool check_digits(int cube, int fourth) {  
    int count[10] = {0}; // 数字0-9的计数数组
```

```
    // 统计立方数的数字
```

```
    int temp = cube;
```

```
    while (temp > 0) {
```

```
        count[temp % 10]++;
```

```
        temp /= 10;
```

```
    }
```

```
    // 统计四次方数的数字
```

```
    temp = fourth;
```

```
    while (temp > 0) {
```

```
        count[temp % 10]++;
```

```
        temp /= 10;
```

```
    }
```

```
    // 检查是否每个数字恰好出现一次
```

```
    for (int i = 0; i < 10; i++) {
```

```
        if (count[i] != 1) {
```

```
            return false;
```

```
        }
```

```
    }
```

```
    return true;
```

```
}
```

4.单词接龙

小明和小刚玩单词接龙游戏。一人先写一个英文单词，然后另一个人回复一个英文单词，要求回复单词的开头有若干个字母与上一个人所写的单词的结尾若干个字母相同，重合部分的长度不限。例如，happy和python的重合部分是py。现在，小明回复了小刚一个单词，小刚想要通过编程来找出这个单词与自己发过去的那个单词的重合部分是什么。请帮助小明完成任务。

注：strncmp函数是C语言标准库中的一个函数，用于比较两个字符串的前n个字符是否对应相等，或者在字典顺序上哪一个更小，若相等则返回0。本题要求不直接调用这个现有函数，而是自行编程实现。

遍历可能的重叠长度，从两个单词长度的最小值开始递减。

对于每个长度，比较第一个单词的末尾子串和第二个单词的开头子串。

如果匹配，则输出该子串作为重叠部分。

如果没有找到任何重叠，则输出空字符串。

```
void find_max_overlap(const char *a, const char *b, char *overlap) {
    int len_a = strlen(a);
    int len_b = strlen(b);
    int max_possible = len_a < len_b ? len_a : len_b; // 最大可能重叠长度

    // 从大到小尝试所有可能的重叠长度
    for (int k = max_possible; k > 0; k--) {
        int match = 1; // 假设匹配
        // 比较 a 的后 k 个字符与 b 的前 k 个字符
        for (int i = 0; i < k; i++) {
            if (a[len_a - k + i] != b[i]) {
                match = 0; // 出现不匹配字符
                break;
            }
        }
        if (match) {
            // 找到最大重叠，复制到 overlap
            strncpy(overlap, a + len_a - k, k);
            overlap[k] = '\0'; // 确保字符串结束
            return;
        }
    }
    // 没有找到任何重叠
    overlap[0] = '\0';
}
```

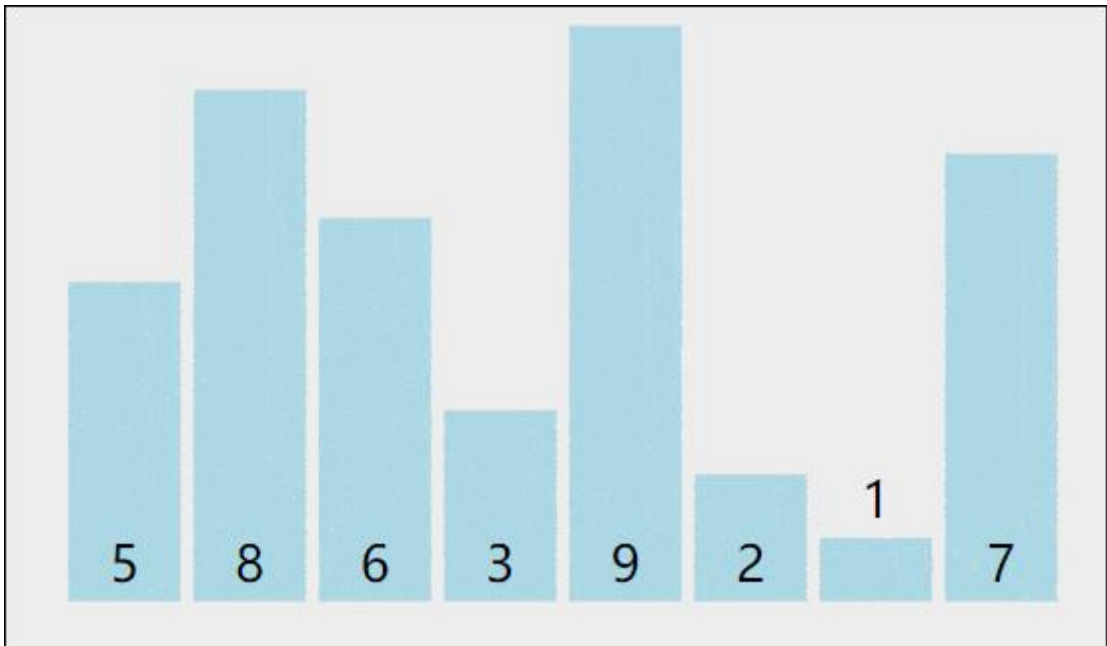
冒泡排序

它重复地“遍历”要排序的列表，一次比较两个相邻的元素，如果它们的顺序错误（比如前一个比后一个大）就交换它们。这个过程就像水底的气泡逐渐上浮到水面一样，较小的（或较大的）元素会经过交换慢慢“浮”到数列的顶端，因此得名“冒泡排序”。

步骤：

- 1.多次遍历数组，比较相邻元素。
- 2.若顺序错误则交换，直到无交换发生为止。

```
void bubble_sort(int arr[], int len) {  
    for (int i = 0; i < len - 1; i++) {  
        int flag = 1;  
        for (int j = 0; j < len - 1 - i; j++) {  
            if (arr[j] > arr[j + 1]) {  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
                flag = 0;  
            }  
        }  
        if (flag) break;  
    }  
}
```



特点：

稳定，时间复杂度 $O(n^2)$ ，适合小规模数据。

选择排序



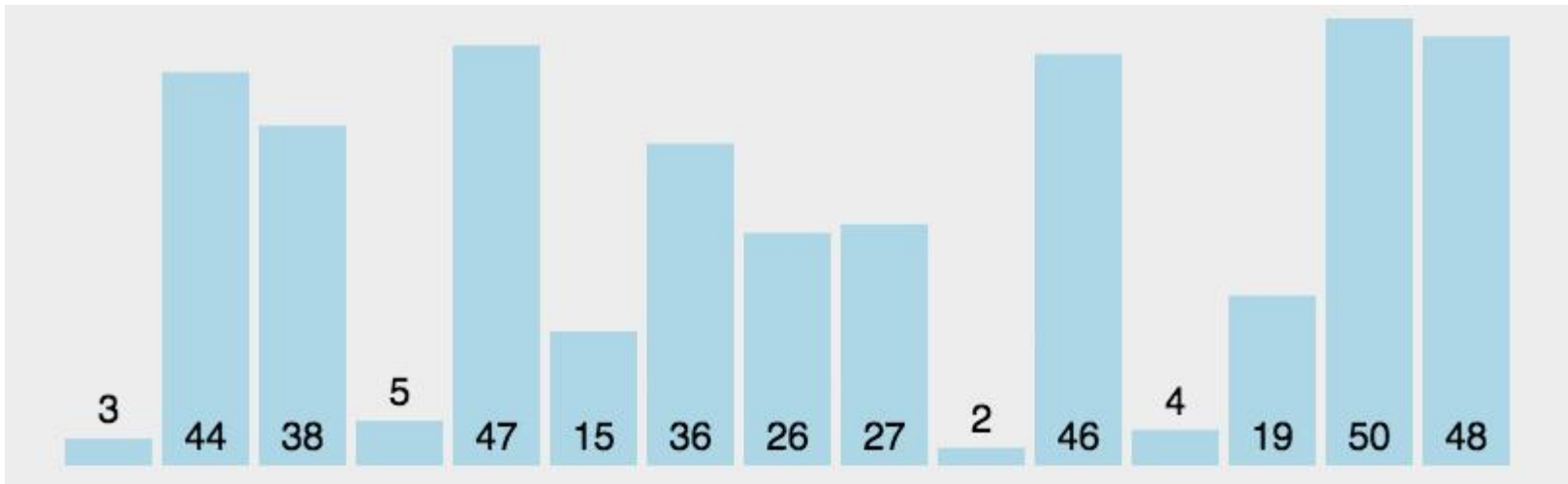
步骤：

每轮从未排序部分选出最小值。

与当前轮起始位置交换。

```
void selection_sort(int a[], int len) {  
    for (int i = 0; i < len - 1; i++) {  
        int min = i;  
        for (int j = i + 1; j < len; j++)  
            if (a[j] < a[min]) min = j;  
        if (min != i) {  
            int temp = a[min];  
            a[min] = a[i];  
            a[i] = temp;  
        }  
    }  
}
```

特点： 不稳定，时间复杂度 $O(n^2)$ ，交换次数少。



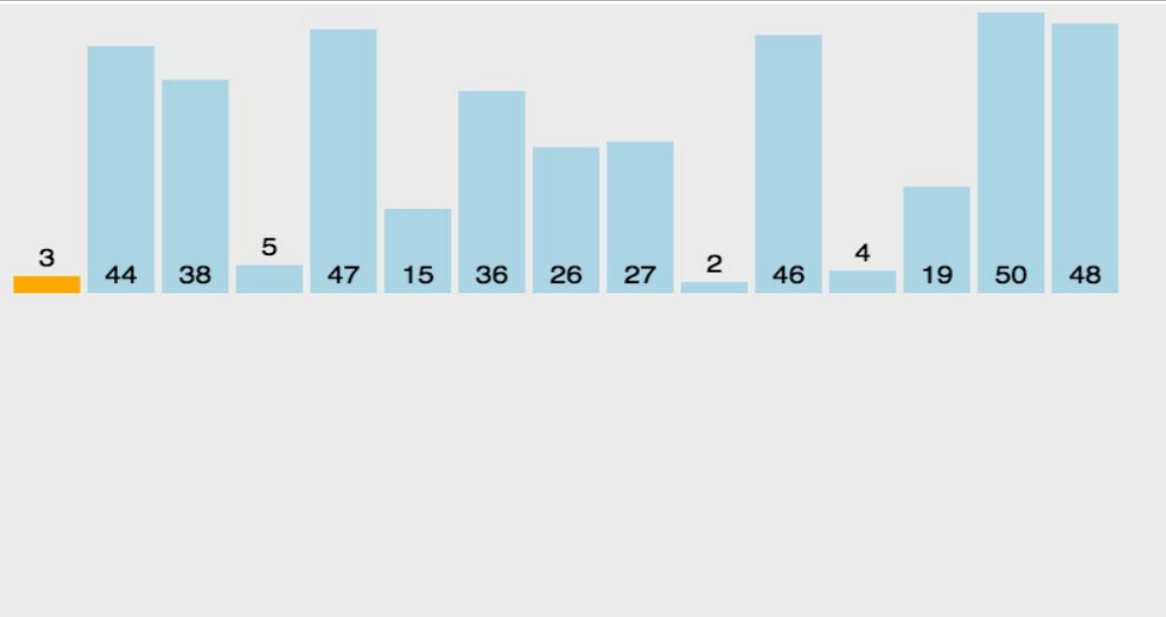
插入排序（Insertion Sort）

插入排序（Insertion-Sort）的算法描述是一种简单直观的排序算法。它的工作原理是通过构建有序序列，对于未排序数据，在已排序序列中从后向前扫描，找到相应位置并插入。

3.1 算法描述

一般来说，插入排序都采用in-place在数组上实现。具体算法描述如下：

- 从第一个元素开始，该元素可以认为已经被排序；
- 取出下一个元素，在已经排序的元素序列中从后向前扫描；
- 如果该元素（已排序）大于新元素，将该元素移到下一位置；
- 重复步骤3，直到找到已排序的元素小于或者等于新元素的位置；
- 将新元素插入到该位置后；
- 重复步骤2~5。



```
void InsertionSort(int *arr, int size)
{
    int i, j, tmp;
    for (i = 1; i < size; i++) {
        if (arr[i] < arr[i-1]) {
            tmp = arr[i];
            for (j = i - 1; j >= 0 && arr[j] > tmp; j--) {
                arr[j+1] = arr[j];
            }
            arr[j+1] = tmp;
        }
    }
}
```


C语言流程图画法

课本27页

符号名称	图形样式	在C语言中的含义与应用
起止框	椭圆形	表示程序的开始（如 main函数入口）和结束（如 return 0） ³ 。
处理框	矩形	表示一个或多个操作步骤，如变量赋值、算术运算、函数调用等 ³ ⁶ 。
判断框	菱形	表示条件判断，对应 if、if-else、switch等选择语句，通常有“是”（Y）和“否”（N）两个出口 ³ ⁶ 。
输入/输出框	平行四边形	表示数据的输入（如 scanf）和输出（如 printf）操作 ³ ⁷ 。
流程线	带箭头的直线	指示程序执行的流向，标准方向为 从上到下、从左到右 ，非标准方向必须画箭头 ⁵ 。

确定程序逻辑

在开始绘制流程图之前，先明确程序的逻辑结构，例如顺序结构、选择结构（如if-else）、循环结构（如while或for）。逐步绘制与检查遵循从上到下、从左到右的原则逐步绘制：

- 放置起止框：从“开始”符号开始。
- 添加步骤：按顺序添加处理、输入/输出框。
- 描绘分支与循环：使用判断框清晰表达条件分支。
- 连接所有符号：用带箭头的流程线连接，确保流向清晰。
- 最终以结束框收尾。
- 绘制完成后，务必仔细检查，确保逻辑正确、流程连贯、没有遗漏。

该输入什么，也能知道程序执行到了哪条语句，以方便进行程序的调试。这些 printf 语句由于不需要输出数据，因此没有格式字符“%”，此时会输出“”中的全部字符串内容。

以上程序编译运行后，会先提示“输入体重（千克）：”，等待输入体重值后，接着提示“输入身高（米）：”，接收到身高值后，立刻计算出 bmi 的值，并按照条件判断结果，输出“你超重了哦！”或“你没有超重。”

包含分支语句的程序会随条件不同而改变执行语句，虽然简单，也可以称得上是算法了，可以用流程图进行算法的描述。流程图是使用最早的算法流程与程序流程描述工具。国家标准中的常用流程图符号如图 2-1 所示，具体的符号用法可以在后续的示例中学习。



图 2-1 常用流程图符号

针对上例程序，完整的流程图如图 2-2 所示。

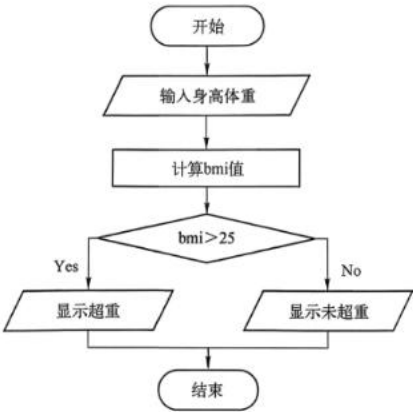


图 2-2 流程图示例

规范的流程图的画法中应包含起止框，并且用平行四边形描述程序的输入和输出。矩形块中描述处理过程，菱形块执行分支条件的判断，务必注意每个分支前要加上 Yes 或 No 的判断结果。显然，起止框只能单向流入或流出，其他类型的框都应该既有流入又有流出，而判断框虽然可以有多个流出，但只能根据判断条件选择其中之一流出。不过也能看到，一些简略的画法中省略了起止框，并用矩形块描述输入输出。

刚开始学习流程图时读者可能会觉得很麻烦，明明只有几行简单的语句，却要花费很长时间去画流程图，有必要吗？其实，绘制基本语句的流程图只是一种训练手段，流程图最常见的应用场景是描述程序的总体流程或算法逻辑。比如，图 2-2 中的“显示超重”和“显示未超重”，在实际的程序中可能分别是“执行减肥计划”与“执行健身计划”。这样其他程序员或项目管理者不需要查阅程序就能清楚地知道这个程序的逻辑结构，便于进一步的交互。

除此以外，流程图也是很直观的算法描述工具。在编写程序前，用流程图描述算法逻辑有

算法特征

- 有穷性：包含有限的操作步骤，不能无限制的执行下去
- 可行性：算法中的每一条指令必须是切实可执行的。
- 确定性：算法中的每一条指令必须有确切的含义，不能产生歧义。

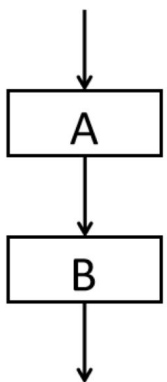
for循环

for循环形式：for (表达式1赋值语句；表达式2判断语句；表达式3自加自减语句)

流程图：

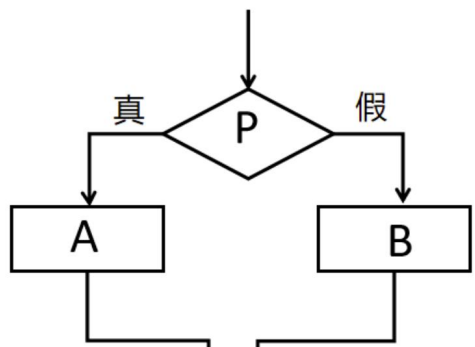


顺序结构



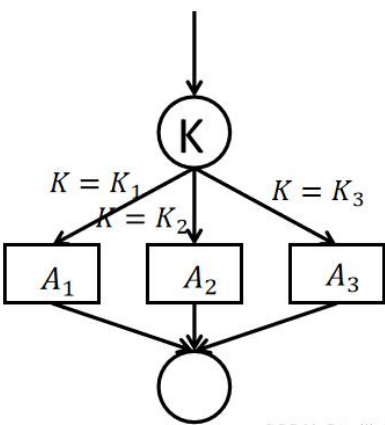
流程图

选择结构

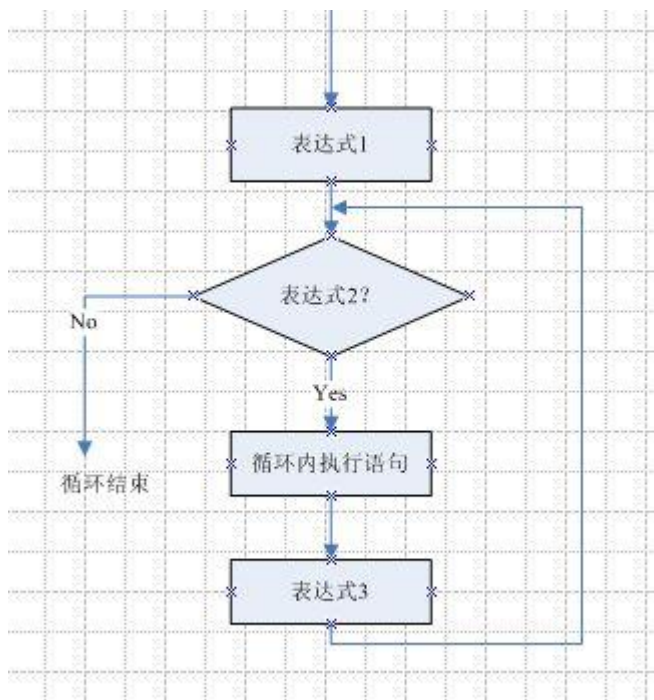


流程图

多分支选择结构



CSDN @twilight_469

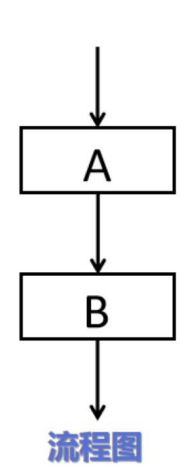


算法特征

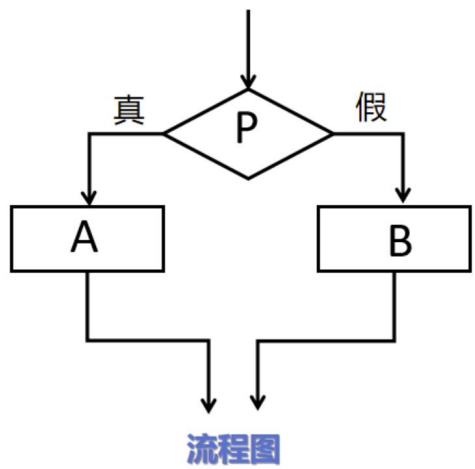
- 有穷性：包含有限的操作步骤，不能无限制的执行下去
- 可行性：算法中的每一条指令必须是切实可执行的。
- 确定性：算法中的每一条指令必须有确切的含义，不能产生歧义。



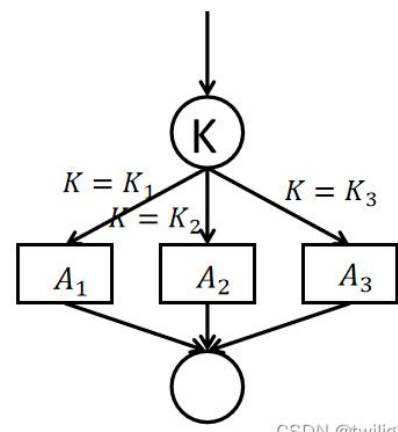
顺序结构



选择结构



多分支选择结构

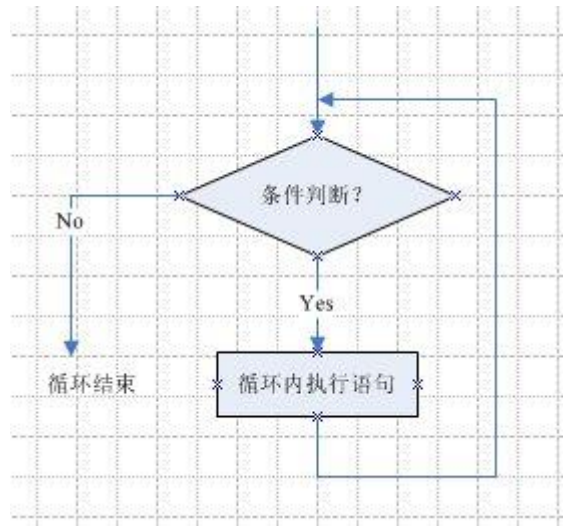


while 循环

while 循环形式：

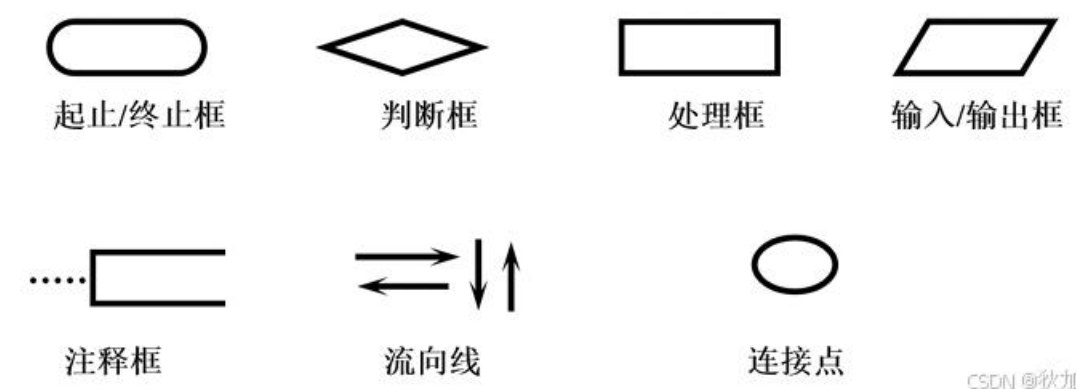
```
while (判断条件)
{
    执行语句;
}
```

流程图：

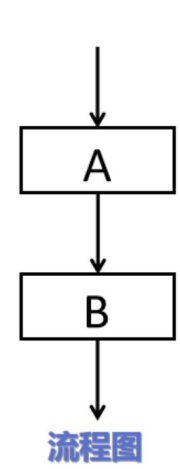


算法特征

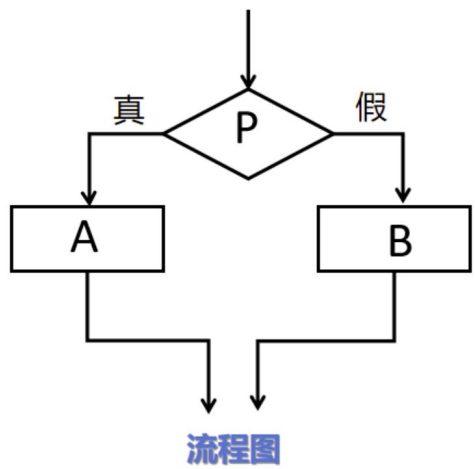
- 有穷性：包含有限的操作步骤，不能无限制的执行下去
- 可行性：算法中的每一条指令必须是切实可执行的。
- 确定性：算法中的每一条指令必须有确切的含义，不能产生歧义。



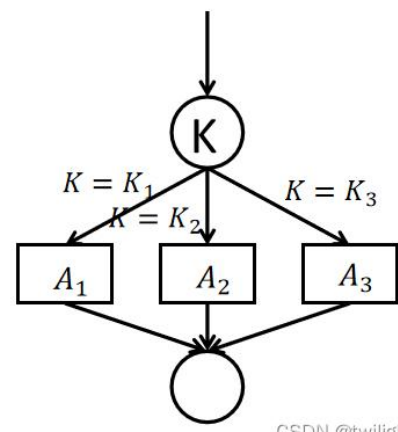
顺序结构



选择结构



多分支选择结构

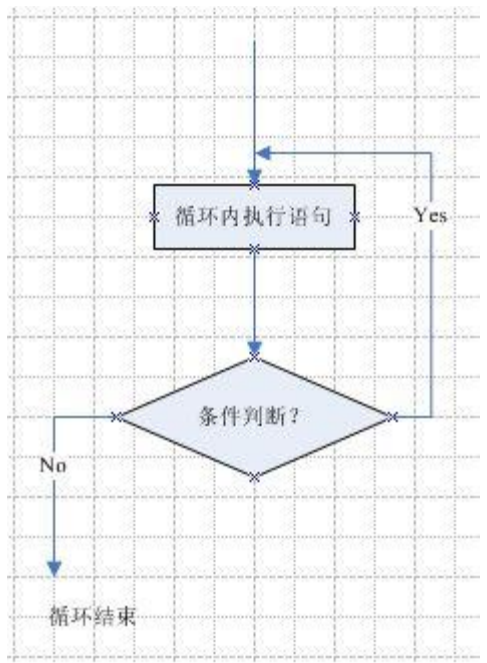


do-while 循环

do-while 循环形式：

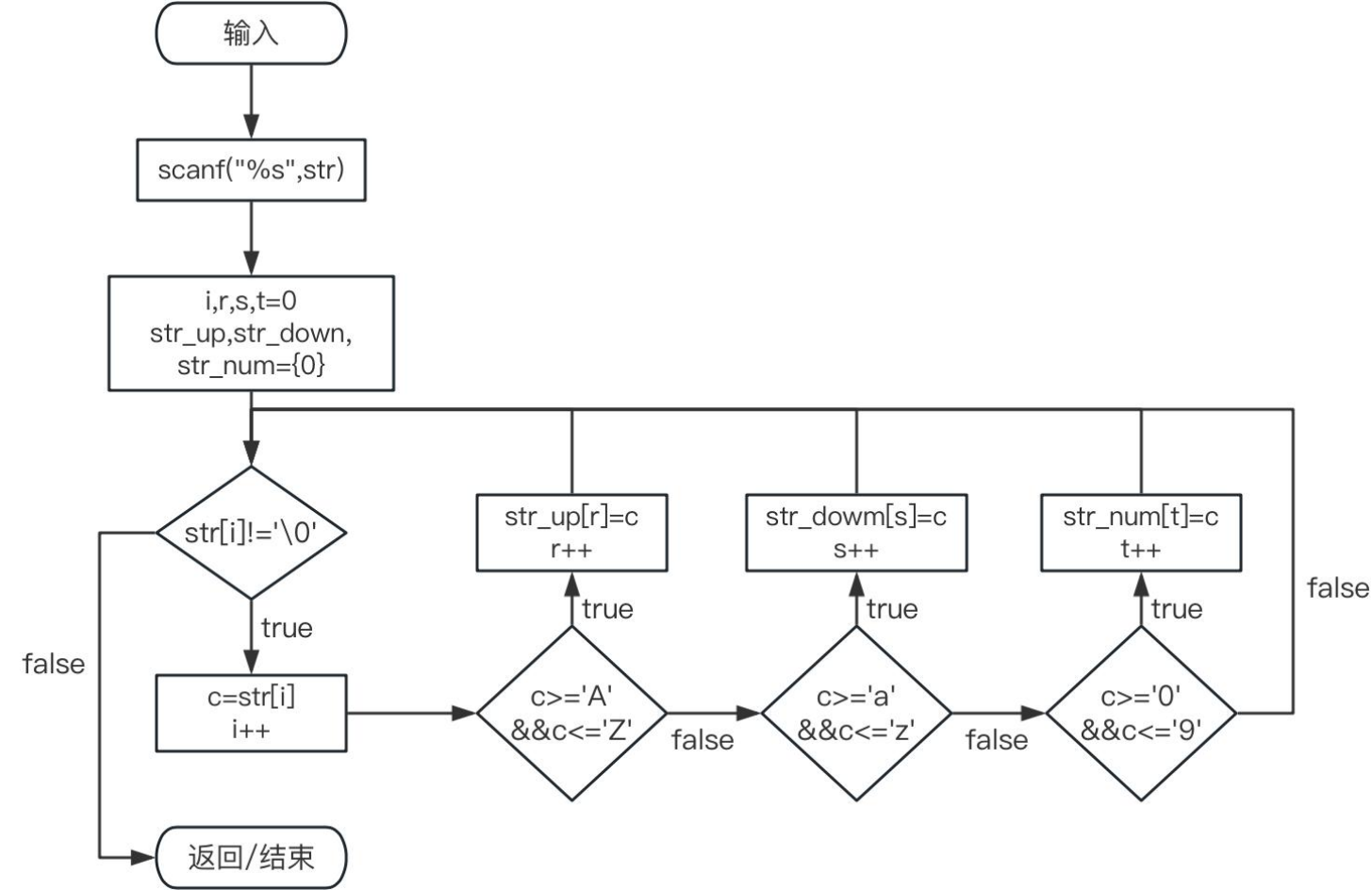
```
do
{
    执行语句;
}while (条件判断)
```

do-while 循环流程图：



[未命名绘图 - draw.io](#) [一个在线绘制流程图的网站](#)

1、从键盘输入一个字符串，其中包含大写字母、小写字母和数字，要求将他们分开并存储到不同的数组中。请用流程图描述编程求解算法。



伪代码是描述算法过程的一种语言，它可以直接用中文或者英文等自然语言表示，也可以直接上代码，但常用经典的语法表示(讲的就是这种)。日常生中,别人能通过你的描述明白相应算法就行。（这个最主要的是解决每个人擅长的程序语言不同，我们不同工程师之间要进行交流，实现代码跨平台转移或者对算法一起进行思考，就需要一种通用且没有歧义 的表示，于是伪代码孕育而生）

伪代码和C语言长得很相似，在某种意义上，不过它又不完全一样

基本特点 语言中立：不依赖特定编程语言;逻辑清晰：关注算法逻辑而非语法细节;易读易懂：使用自然语言和简单结构 层次分明：通过缩进表示程序结构

伪代码示例

以下是冒泡排序的伪代码示例：

```
Procedure BubbleSort(A: array of integer, n: integer)
  Begin
    For i := 1 To n-1 Do
      Begin
        For j := n-1 DownTo i Do
          If A[j] > A[j+1] Then
            Swap(A[j], A[j+1]);
          End;
        End BubbleSort
      End
```

在这个示例中，我们定义了一个名为 BubbleSort 的过程，它接受一个整数数组 A 和一个整数 n 作为参数。算法的主体部分是两个嵌套的 For 循环，用于比较和交换数组中的元素。

伪代码的基本结构

• 算法名称：伪代码通常以算法名称开头，可以是过程（Procedure）或函数（Function）。过程不返回值，而函数返回值。例如：

```
Procedure AlgorithmName(parameters)
Function AlgorithmName(parameters)
```

• 指令序列：指令序列是算法的主体部分，用于描述算法的具体步骤。可以使用 Begin 和 End 标记开始和结束，或者使用大括号 {}。例如：

```
Begin
  指令序列;
End
```

• 输入/输出：输入使用 Input，输出使用 Output 或 Return。例如：

```
Input: x, y
Output: result
```

• 分支选择：使用 If 和 Else 语句来表示条件分支。可以嵌套使用。例如：

```
If condition Then
  指令序列1;
Else
  指令序列2;
```

• 赋值：使用 := 或 <- 表示赋值操作，相当于C++中的 =。例如：

```
x := x + 1;
```

• 循环：包括计数式循环（For循环）和条件式循环（While循环）。例如：

```
For i := 1 To n Do
  指令序列;
```

• 算法结束：使用 End 关键字加上算法名称表示算法结束。例如：

```
End AlgorithmName
```

2、有一个函数，其功能为：将一个无序数组 a 中的每个元素都插入到一个降序数组 b 中，并且使 b 为降序排列。请用类 C 伪代码写出此函数。

两种思路

1.将a全部添加到b中，
进行降序排列

2.将a的元素挨个插入
b中，插入时注意排序

```
def descending_sort(a, b, N=len(a), M=len(b)):
    for i in range(N): # i = 0 -> N-1
        for j in range(M+1): # j = 0 -> M
            if j==M or a[i]>b[j]:
                b.append(0) # 拓展b的长度
                for k in range(M-1, j-1, -1): # k = M-1 -> j
                    b[k+1] = b[k]
                b[j] = a[i]
                M += 1
                break
    return 0
```

第1题，P48 1求 π 的近似值

思路：选择一种 π 的级数展开式作为代码实现的基础；

根据要求输入迭代次数以及数据类型；

构造循环以及case语句实现具体 π 近似值的计算；

按照不同的数据类型输出结果。

```
switch(t){//根据选择的类型输出计算结果
    case 1: printf("迭代%u次，pi近似为: %.8f\n",n,f_pi);
             break;
    case 2: printf("迭代%u次，pi近似为: %.16f\n",n,d_pi);
             break;
    case 3: __mingw_printf("迭代%u次，pi近似为: %.20Lf\n",n,ld_pi);
             break;
}
return 0;
}
```

```
#include <stdio.h>
```

```
int main()//求pi的近似值
```

```
{
```

```
    unsigned int i,n=100;//n-迭代次数
```

```
    unsigned int t=1;//选择数据类型: 1-float,2-double,3-long double
```

```
    float f_pi=2.0,f_i=1.0;//用float类型计算,精确到小数点后6位
```

```
    double d_pi=2.0,d_i=1.0;//用double类型计算,精确到小数点后15位
```

```
    long double ld_pi=2.0,ld_i=1.0;//用long double类型计算,精确到小数点后18位
```

```
    printf("输入计算的迭代次数(0~4294967295):");//提示
```

```
    scanf("%u",&n);//输入一个无符号整数到n
```

```
    printf("选择数据类型:1-float,2-double,3-long double: ");//提示选择数据类型
```

```
    scanf("%u",&t);//输入一个无符号char(1字节整数)到t
```

```
    for(i=1;i<=n;i++)//填空: 计算n次
```

```
    {
```

```
        switch (t) //根据选择的类型进行计算
```

```
        {
```

```
            case 1:
```

```
                f_pi *=4.0*f_i*f_i/(2*f_i-1)/(2*f_i+1);//float
```

```
                f_i+=1.0;
```

```
                break;
```

```
            case 2:
```

```
                d_pi *=4.0*d_i*d_i/(2*d_i-1)/(2*d_i+1);//填空: double
```

```
                d_i+=1.0;
```

```
                break;
```

```
            case 3:
```

```
                ld_pi *=4.0*ld_i*ld_i/(2*ld_i-1)/(2*ld_i+1);//long double
```

```
                ld_i+=1.0; //填空
```

```
                break;
```

```
        }
```

```
    }
```

第2题，用计算机模拟求解方法（蒙特卡洛法）求 π 的近似值

```
#include <stdio.h>
#include <stdlib.h>
#include<math.h>
#include<time.h>
No2(){ //用计算机模拟求解方法（蒙特卡洛法）求 $\pi$  的近似值，参见参考资料
    long k=0,c=0,d=0;
    double pai=0,x=0,y=0;

    srand((unsigned int)time(NULL));
    for (k=0;k<100000;k++){
        d++;
        x=(double)rand()/32767.0;
        y=(double)rand()/32767.0;

        if(sqrt(x*x+y*y)<1.0){
            c++;
        }
    }
    pai=4.0*c/d;
    printf("π 的近似值是: %18.14lf\n",pai);
}
```


一、rand()

rand()函数用来产生随机数，但是，rand()的内部实现是用线性同余法实现的，是伪随机数，由于周期较长，因此在一定范围内可以看成是随机的。rand()会返回一个范围在0到RAND_MAX（至少是32767）之间的伪随机数（整数）。

在调用rand()函数之前，可以使用srand()函数设置随机数种子，如果没有设置随机数种子，rand()函数在调用时，自动设计随机数种子为1。随机种子相同，每次产生的随机数也会相同。

rand()函数需要的头文件是：<stdlib.h>

rand()函数原型：int rand(void);

使用rand()函数产生1-100以内的随机整数：int number1 = rand() % 100+1;

二、srand() srand()函数需要的头文件仍然是：<stdlib.h>

srand()函数原型：void srand (unsigned int seed);

srand()用来设置rand()产生随机数时的随机数种子。参数seed是整数，通常可以利用time(0)或getpid(0)的返回值作为seed。

使用rand()和srand()产生1-100以内的随机整数：

srand(time(0));

int number1 = rand() % 100+1;

三、使用rand()和srand()产生指定范围内的随机整数的方法

“模除+加法”的方法

因为，对于任意数， $0 \leq \text{rand()} \% (n-m+1) \leq n-m$

因此， $0+m \leq \text{rand()} \% (n-m+1) + m \leq n-m+m$

因此，如要产生[m,n]范围内的随机数num，可用：

int num=rand()%(n-m+1)+m;

其中的rand()%(n-m+1)+m算是一个公式，记录一下方便以后查阅。

比如产生10~30的随机整数：

srand(time(0));

int a = rand() % (21)+10;

1. 关于C语言程序，以下叙述中正确的是（ ）

A) main函数必须位于所有其他函数之前

B) 预处理命令属于特殊的C语言语句

C) 每条语句独占一行，换行表示语句结束

D) C语言的输入和输出功能只能通过函数调用实现

D

2. 下列选项中，错误的转义字符是（ ）。

- A) '\n'
- B) '\x78'
- C) '\78'
- D) '\"'

C

3. 设有float x=5；则表达式(++x) + 1/2的值为（ ）。

- A) 5.0
- B) 5.5
- C) 6.0
- D) 6.5

答案：C) 6.0

解析：

1. ++x：这是前缀增量运算符。x的值先自增1，从 5.0 变为 6.0，然后表达式使用新值 6.0。

2. 1/2：因为 1和 2都是整数，所以进行的是整数除法，结果为 0。

3. 整个表达式为：6.0 + 0，结果为 6.0（浮点数类型）。

解析：

- A) 错误。main函数是程序的入口，但它在程序中的位置可以是任意的。它可以位于其他函数之前、之后或之间。编译器会找到 main函数作为起点。
- B) 错误。预处理命令（如 #include, #define）在编译之前由预处理器处理，它们不是真正的C语言语句，不以分号结尾。
- C) 错误。C语言以分号 ;作为语句的结束标志。一条语句可以写成多行，多条语句也可以写在一行内（尽管不推荐）。
- D) 正确。C语言本身没有输入输出语句，其输入输出功能是通过调用标准库函数（如 printf, scanf）来实现的。

A) '\n'：正确，表示换行符。

B) '\x78'：正确，\x后跟1-2位十六进制数，表示该ASCII码对应的字符。'\x78'对应字符 'x'。

C) '\78'：错误。'\78'后跟1-3位八进制数表示一个字符，但八进制数只能包含数字0-7。这里的 '8' 是无效的八进制数字。

D) '\"'：正确，表示双引号字符 "。

课本42页

判断条件是 $c >= 'a' \& \& c <= 'z'$ ，此时变量 c 的内容作为小整数使用，其中存放的值分别与常量 $'a'$ 和 $'z'$ 的 ASCII 值进行关系运算，可翻译为“变量 c 中存放的字符是否在从 a 到 z 的字母表中”，这种用法强烈建议牢记于心。

运行修改后的程序，从键盘连续输入“a2c@e”5 个字符后回车，可以得到运行结果：

```
a2c@e
A2C@E
```

字符常量的一对“ $'$ ”中通常只包含一个字符，如字母（ $'z'$ ）、数字（ $'8'$ ）、标点符号（ $'@'$ ）等，但为了书写一些不可打印的字符或者非十进制的数字，会在这些字符或数字前加上“ $\backslash$ ”构成转义字符。转义字符有如下形式：

❖ “ $\backslash$ ”后跟 1 个字符。如 $\backslash n$ 表示换行， $\backslash t$ 表示 tab， $\backslash \backslash$ 表示字符“ $\backslash$ ”，而为输出“和”，应写成 $\backslash "$ 和 $\backslash "'$ 。

❖ “ $\backslash$ ”后跟 1~3 位八进制数。如 $\backslash 12$ 或 $\backslash 012$ 是其八进制数代表的字符，在 ASCII 表中应换行符，即与 $\backslash n$ 等效。

❖ “ $\backslash$ ”后跟 1~2 位 x （不能用 X ）引导的十六进制数。如 $\backslash xA$ 是其十六进制数所代表的字符，在 ASCII 表中对应换行符，仍与 $\backslash n$ 等效。

转义字符列表如表 3-1 所示，在使用时学习即可，不需专门记诵。

表 3-1 常用的转义字符

转义字符	ASCII 值	功 能
$\backslash a$	7	响铃 (beep)
$\backslash b$	8	退格符，打印位置向左移一列
$\backslash f$	12	走纸换页，打印位置移到下一页的开头
$\backslash t$	9	横向制表符，打印位置移到下一个横向制表位
$\backslash n$	10	换行符，打印位置移到下一行的开头
$\backslash r$	13	回车符，打印位置移到当前行的开头
$\backslash v$	11	纵向制表符，打印位置移到下一个纵向制表位
$\backslash \backslash$	92	反斜杠字符 $\backslash$
$\backslash ?$	63	问号字符 $?$
$\backslash '$	39	单引号字符 $'$
$\backslash "$	34	双引号字符 $"$
$\backslash o$, $\backslash oo$, $\backslash ooo$		1~3 位八进制数，ASCII 值等于该值的字符
$\backslash xh$, $\backslash xhh$		1~2 位十六进制数，ASCII 值等于该值的字符

4. 下列选项中，语句（ ）不能正确表示如下操作：当a大于b时，x自增1。

- A) (a>b) && (x++)
- B) x=(a>b,1)
- C) if(a>b)x++;
- D) x=(a>b)?x+1:x;

5. 若变量已正确定义并赋值，则下列表达式中有语法错误的是（ ）。

- A) (int) a/ (int)b%2
- B) (float) a%b
- C) (int)b/2*c
- D) a/b/c

解析：

- A) (a>b) && (x++)：正确。逻辑与 &&有短路特性。如果 a>b为真，则会执行 x++使 x自增1。
- B) x=(a>b, 1)：错误。这是逗号表达式。逗号表达式的值是最后一个子表达式（即 1）的值。无论 a>b是否成立，x都会被赋值为 1，x的值并未自增。
- C) if(a>b) x++;：正确。标准的条件判断语句。
- D) x=(a>b)? x+1 : x;：正确。三目条件运算符。如果 a>b为真，则将 x+1的值赋给 x，相当于 x自增1。

- A) (int)a / (int)b % 2：正确。先对 a和 b进行强制类型转换为 int，然后进行除法 /和取模 %运算。/和 %的优先级相同，结合性为从左到右。
- B) (float) a % b：错误。取模运算符 %要求左右两个操作数必须都是整型。即使将 a强制转换为 float，%右边的 b也可能是浮点数，这是不合法的。
- C) (int)b / 2 * c：正确。将 b转为 int，然后进行除法和乘法运算。
- D) a / b / c：正确。连续的除法运算。

6、下列用户自定义标识符正确的是 ()

- A) 1_xyz
- B) #ustc
- C) h2o
- D) k,bu

7、下面定义二维数组不正确的是 ()

- A) `int a[2][2] = {{4,6},{5,8}};`
- B) `int a[][] = {{4},{6}};`
- C) `int a[2][] = {4,5,6,7};`
- D) `int a[1][4] = {{1,2,3,4}};`

8、若有 `int x=1, y=2, z=3;` 则表达式 `3 < z > y > x` 的值为 ()

- A) 0
- B) 1
- C) 2
- D) 3

9、以下程序的运行结果是 ()。

```
void main() {
    struct cmplx {
        int x, y;
    } cn[2] = {{1, 3}, {2, 7}};
    printf("%d", cn[0].y / cn[0].x + cn[1].x * cn[1].y);
}
```

A) 1_xyz: 错误, 标识符不能以数字开头

B) #ustc: 错误, 标识符不能包含特殊字符#

C) h2o: 正确, 标识符可以包含字母和数字, 但必须以字母开头

D) k,bu: 错误, 标识符不能包含逗号

课本20页

A) `int a[2][2] = {{4,6},{5,8}};`: 正确, 完整初始化

B) `int a[][] = {{4},{6}};`: 在C99标准中允许, 编译器可自动推断维度

C) `int a[2][] = {4,5,6,7};`: 错误, 二维数组定义时必须指定列数

D) `int a[1][4] = {{1,2,3,4}};`: 正确, 单行数组初始化

解析:

表达式从左到右计算:

1. `3 < z` → `3 < 3` → 0 (假)

2. `0 > y` → `0 > 2` → 0 (假)

3. `0 > x` → `0 > 1` → 0 (假)

最终结果为0

左右结合

3.2.4 表达式的运算次序

包含两个以上运算符的表达式需要通过优先级与结合性（也称结合律、结合方向）规则确定运算符的运算次序。表 3-2 给出了除位运算符以外所有运算符的优先级与结合性，同一行中的运算符具有相同的优先级，各行间从上往下优先级逐行降低。完整的运算符优先级与结合性列表见附录 C。

表 3-2 运算符的优先级与结合性

优先级	运算符	结合性
1	() [] -> . 后缀++ 后缀--	从左向右
2	! ~ ++ -- + - * & (type) sizeof 前缀++ 前缀--	从右向左
3	* / %	从左向右
4	+ -	
6	< <= > >=	
7	= !=	
11	&&	
12		从右向左
13	?:	
14	= += -= *= /= %=	从左向右
15	,	

在大的类别上，一元运算符优先级 > 二元运算符；算术运算符优先级 > 关系运算符 > 逻辑运算符 > 赋值运算符。同类运算符中也有优先级的差异，如乘、除高于加、减，逻辑与高于逻辑或。

表达式中若有多个相同优先级的运算符，则按结合性确定计算次序。结合性分为从右向左和从左向右两种结合方向。除了后缀自增自减，一元运算符都只能从右向左结合；除了赋值运

9、以下程序的运行结果是（ ）。

```
void main() {
    struct cmplx {
        int x, y;
    } cn[2] = {{1, 3}, {2, 7}};
    printf("%d", cn[0].y / cn[0].x + cn[1].x * cn[1].y);
}
```

2025-12-14

2.1.1 数据与数据类型

80 和 1.85 是直接写在程序里的不变的数值，称为常量（constant）。bmi 是存储 BMI 值的内存空间的名字，该空间中的数据允许被改变，称为变量（variable）。注意，C 语言是大小写敏感的，也就是说，BMI 和 bmi 在程序中是两个不同的变量名。习惯上，C 语言程序中多使用小写字母来命名变量，因此本例使用 bmi 而不是 BMI 作为变量名。变量名是一种标识符，有特定的命名规则。

【说明】标识符（identifier）是指用来标识某个实体的一个符号，在不同的应用环境下有不同的含义。在程序设计语言中，标识符用于给变量、函数等命名，以建立起名称与使用之间的关系。标识符由字母、数字和下划线构成，且不能以数字开头。为便于理解程序，通常建议用有意义的单词或缩写来命名标识符。

C 语言中的变量必须先声明后使用。变量声明语句包含变量的类型和名字。C 语言编译器根据变量声明的类型确定给变量分配多大的空间。C 语言提供的基本数据类型包括：

20

解析：

1. 结构体数组初始化：

°cn[0].x = 1, cn[0].y = 3

°cn[1].x = 2, cn[1].y = 7

2. 表达式计算（注意运算符优先级，乘除高于加减）：

cn[0].y / cn[0].x + cn[1].x * cn[1].y

= 3 / 1 + 2 * 7

= 3 + 14

= 17

下列程序，当输入1并回车后，执行后的输出结果是()。

```
#include<stdio.h>
int main( )
{
    int x, a = 0, b = 1;
    scanf("%d", &x);
    switch(x)
    {
        case 0: b++;
        case 1: a++;
        case 2: a++; b++;
    }
    printf("a=%d,b=%d", a, b);
    return 0;
}
```

代码分析：

1.变量初始化：

- int x, a = 0, b = 1;：变量a初始值为0，b初始值为1。

2.输入处理：

- scanf("%d", &x);：输入x的值为1。

3.switch语句执行（由于没有break语句，会发生"case穿透"）：

- x的值为1，因此直接跳转到case 1:执行。
- case 1: a++;：执行a++，a的值从0变为1。
- 由于没有break，程序继续执行case 2:。
- case 2: a++; b++;：执行a++和b++，a从1变为2，b从1变为2。

4.输出结果：

- printf("a=%d,b=%d", a, b);：输出a和b的值。